SQL Server-Execution Plans

Propietario	© Carlos Roberto
Verificación	Verificada
Etiquetas	DBA

1. Preparación del ambiente (01_stn_setup.sql)

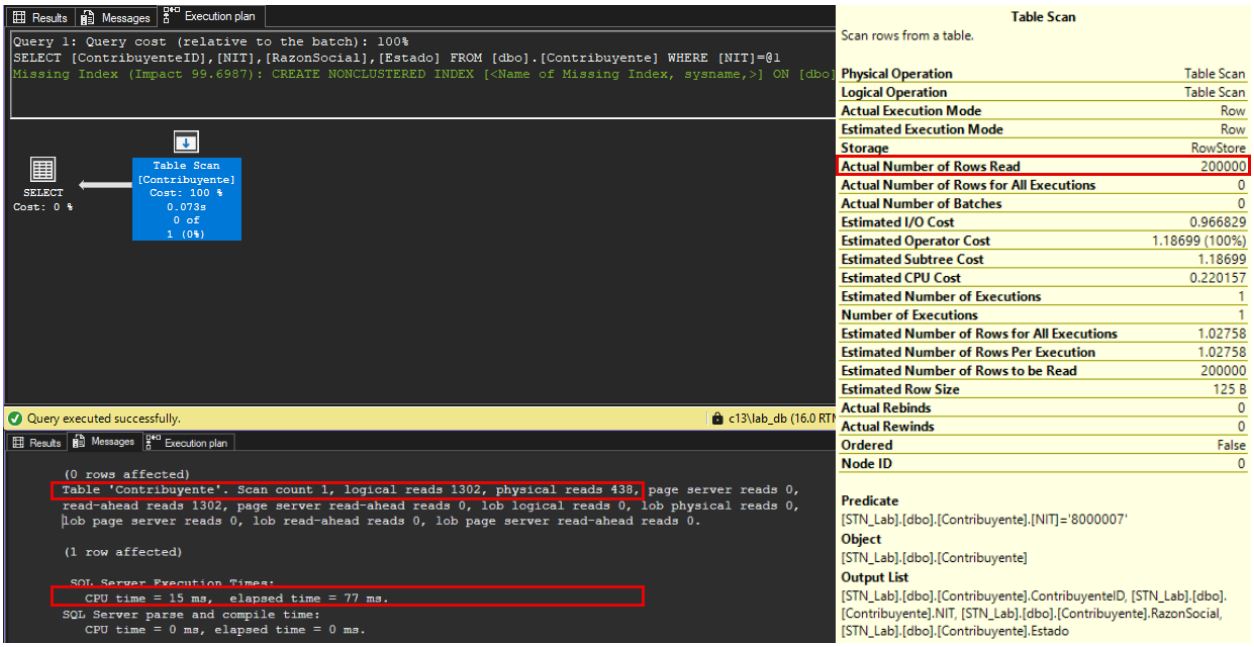
2.1: Table Scan

La búsqueda se realiza sobre toda la tabla, se hace un barrido para tratar de encontrar el resultado de la condición Where.

```
USE STNSRVDB;
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

SELECT ContribuyenteID, NIT, RazonSocial, Estado
FROM dbo.Contribuyente
WHERE NIT = '8000007';
```

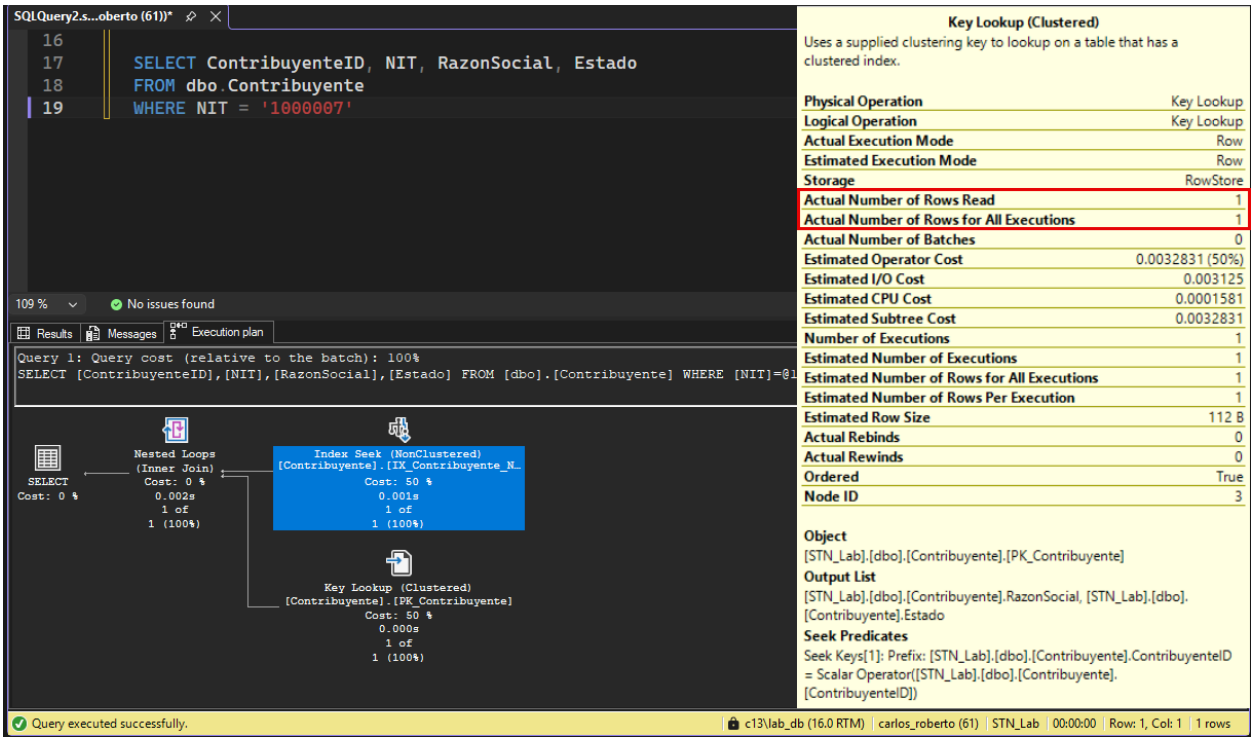


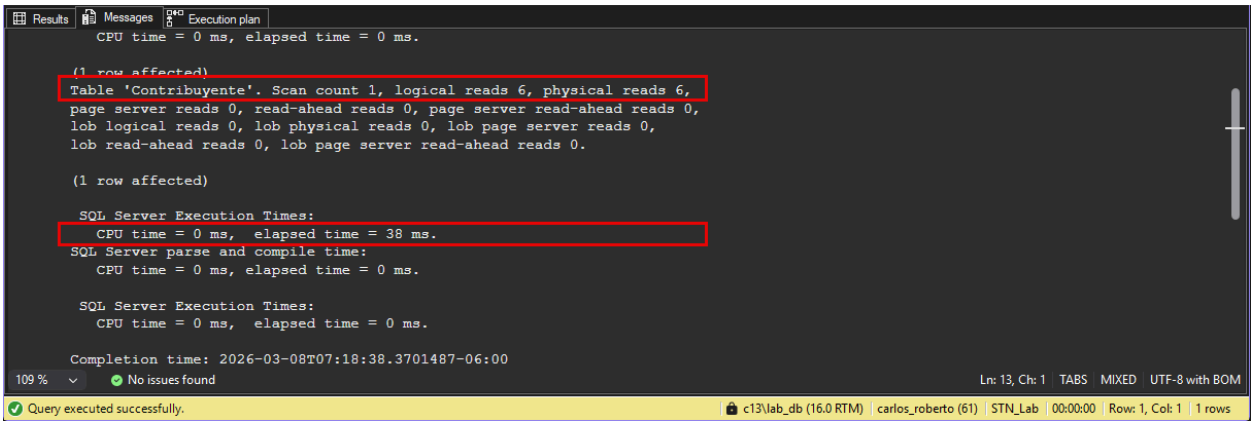
2.2: Index Seek + Key Lookup

Se elimina la Contraint sobre ContribuyenteId (creada al inicio de la operación) y se convierte en un CONSTRAINT PK_Contribuyente PRIMARY KEY CLUSTERED, se agrega índice NONCLUSTERED para una busqueda mas efectiva.

```
ALTER TABLE dbo.Contribuyente DROP CONSTRAINT PK_Contribuyent
e;
GO
ALTER TABLE dbo.Contribuyente
    ADD CONSTRAINT PK_Contribuyente PRIMARY KEY CLUSTERED (Co
ntribuyenteID)
GO
CREATE NONCLUSTERED INDEX IX_Contribuyente_NIT
    ON dbo.Contribuyente (NIT)
GO

SELECT ContribuyenteID, NIT, RazonSocial, Estado
FROM dbo.Contribuyente
WHERE NIT = '8000007'
```





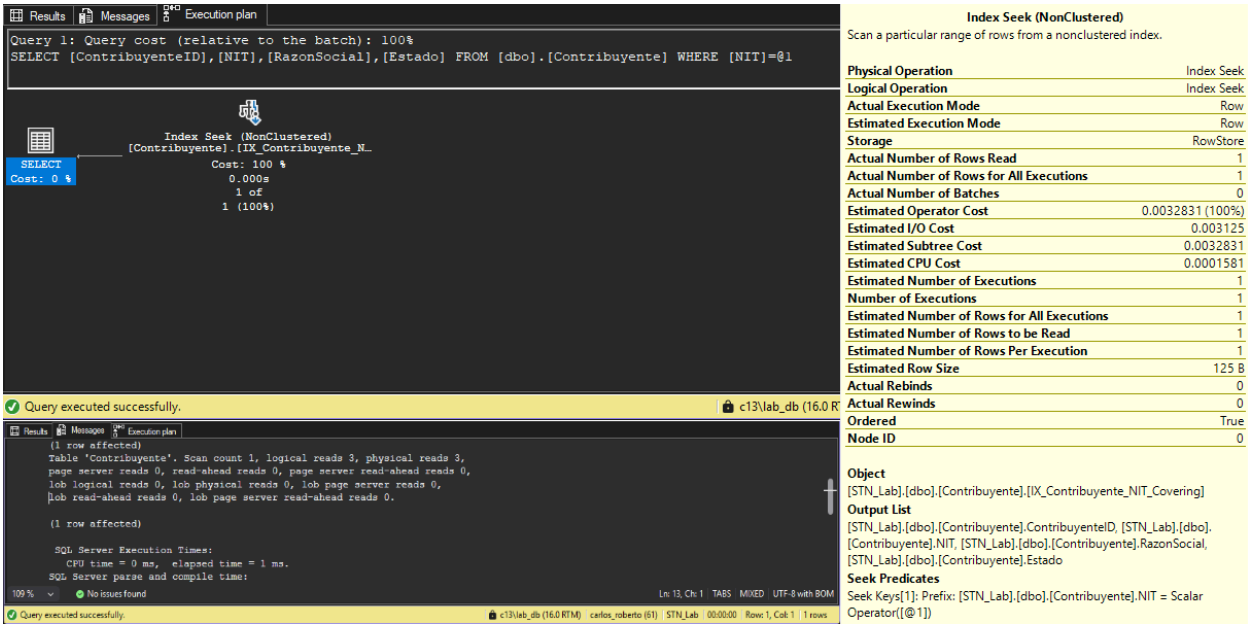
El NIT se encuentra de forma mas efectiva (seek) pero al pedir RazonSocial, Estado, se hace un segundo viaje (Key Lookup) al Clustered para encontrar esos campos.

2.3: Índice de Cobertura

Agregar RazonSocial y Estado al índice con INCLUDE. SQL Server ya no hace el viaje adicional. (Elimina el Key Lookup)

```
CREATE NONCLUSTERED INDEX IX_Contribuyente_NIT_Covering
    ON dbo.Contribuyente (NIT)
    INCLUDE (RazonSocial, Estado);
GO

SELECT ContribuyenteID, NIT, RazonSocial, Estado
FROM dbo.Contribuyente
WHERE NIT = '1000007'
```

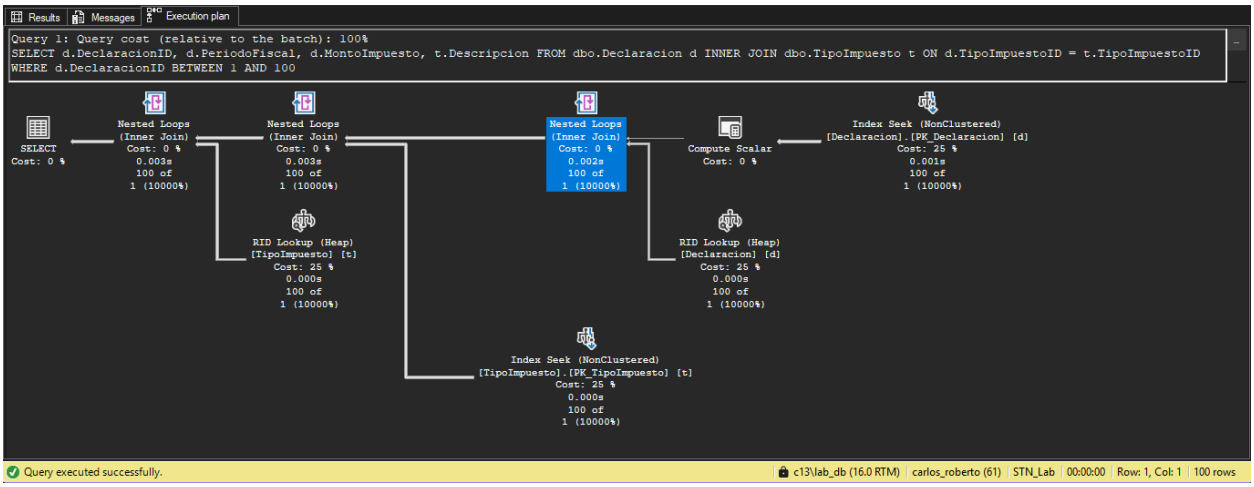


2.4: Tipos de Joins

Los JOINs (INNER JOIN, LEFT JOIN, RIGHT JOIN) son instrucciones lógicas que le dicen a SQL Server qué datos combinar y qué regla usar para combinarlos. Los operadores de JOIN del plan de ejecución son mecanismos físicos. La decisión interna del optimizador sobre cómo ejecutar esa combinación en memoria.

Nested Loops

Por cada fila de la tabla A, busca en la tabla B. Eficiente cuando una tabla es pequeña.



Costo-beneficio:

	Detalle
Cuándo es bueno	Una tabla pequeña + índice disponible en la tabla inner
Cuándo es malo	Tabla outer grande sin filtro — ejecuta N búsquedas donde N = filas de la outer
En este plan	100 declaraciones × 6 tipos de impuesto = operación mínima — decisión correcta

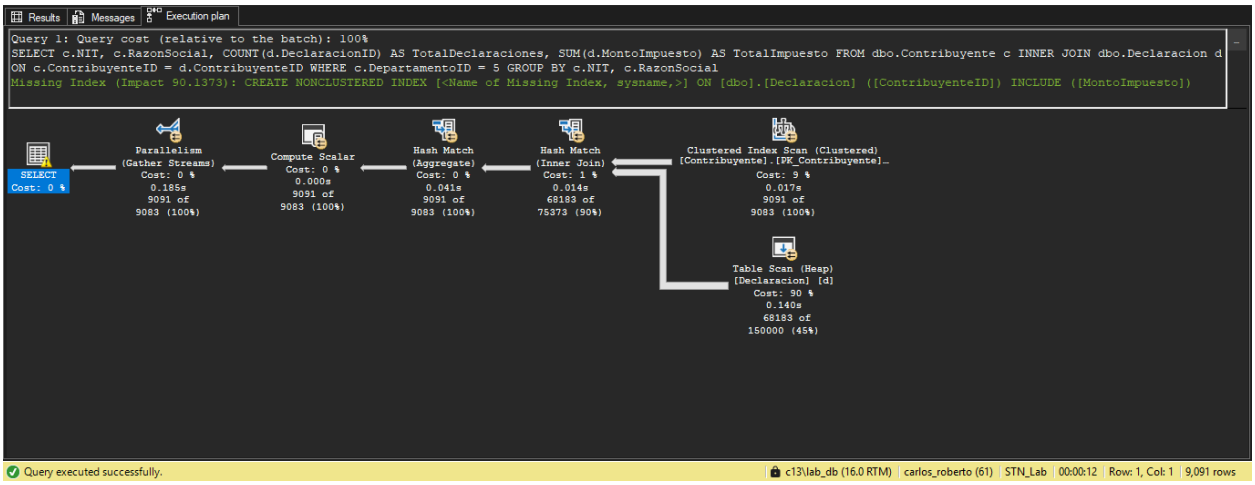
RID Lookup es un operador del plan de ejecución que ocurre cuando SQL Server necesita columnas que no están en el índice que usó para la búsqueda. Al no existir un índice clustered en la tabla, utiliza el Row ID; la dirección física de la fila en disco (archivo + página + posición) para ir a buscar directamente esas columnas faltantes.

Hash Match

Es el operador físico que SQL Server usa para ejecutar un JOIN cuando las tablas involucradas son grandes y no hay un índice disponible en la columna de unión. Construye una tabla hash en memoria con una de las fuentes y luego compara la otra contra ella fila por fila.

```
SELECT
    c.NIT,
```

```
    c.RazonSocial,
    COUNT(d.DeclaracionID)      AS TotalDeclaraciones,
    SUM(d.MontoImpuesto)        AS TotalImpuesto
FROM dbo.Contribuyente c
    INNER JOIN dbo.Declaracion d ON c.ContribuyenteID = d.Con
tribuyenteID
WHERE c.DepartamentoID = 5
GROUP BY c.NIT, c.RazonSocial;
```



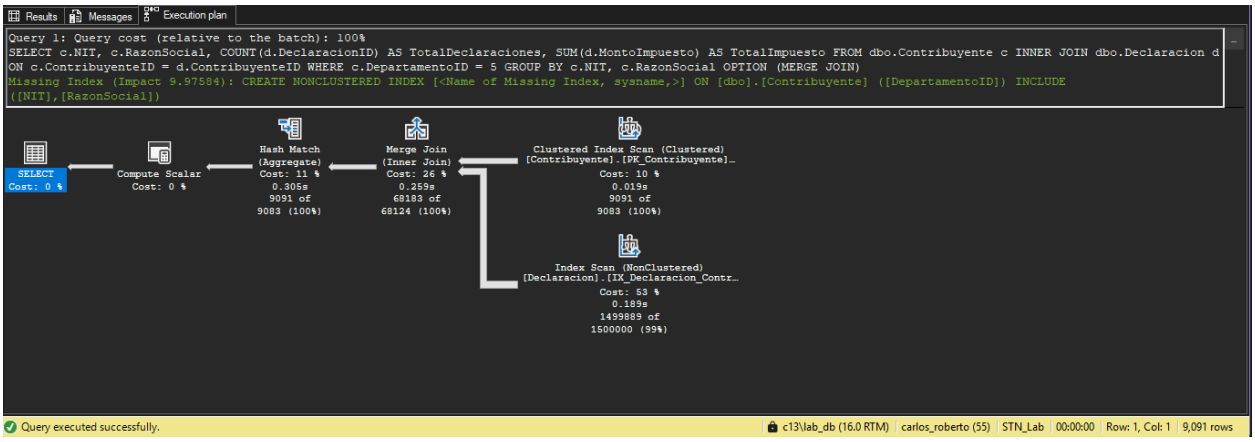
Costo-beneficio:

	Detalle
Cuándo es bueno	Tablas grandes sin índice en el join key — es la mejor opción disponible
Cuándo es malo	Cuando la memoria asignada no alcanza → derrama a TempDB (Spill)

Merge Join

Es el operador físico que SQL Server usa para unir dos fuentes que llegan ordenadas por la misma columna de unión. Lee cada fuente una sola vez de

principio a fin y las combina en un único paso lineal sin construir estructuras en memoria ni realizar búsquedas adicionales por cada fila.



Justificación del Index Scan en este contexto:

El Merge Join exige que ambas fuentes lleguen ordenadas por ContribuyenteID. Para cumplir esa condición sobre 1,500,000 filas en Declaracion, SQL Server debe leer el índice completo, no existe un filtro selectivo que reduzca el volumen antes de ordenar. El Index Scan no es un problema de diseño aquí, es el costo obligatorio que paga el Merge Join para poder operar. Leer el índice completo sigue siendo más barato que leer las páginas de datos completas como lo haría un Table Scan.